

Searching the OMOP Vocabulary with Capr

Contents

0.1	Connecting to a vocabulary	1
0.2	<code>searchConcepts()</code> — keyword search	1
0.3	<code>rankedSearchConcepts()</code> — ranked search	2
0.4	<code>getConceptDescendants()</code> — concept hierarchy	3
0.5	<code>mapSourceToStandard()</code> — source code lookup	3
0.6	<code>getConceptInfo()</code> — lookup by ID	4
0.7	End-to-end: search → build concept set → build cohort	4

`library(Capr)`

Capr provides a set of vocabulary search functions that query the OMOP concept tables directly. They require a live connection to a database that contains the OMOP vocabulary schema (concept, concept_ancestor, concept_relationship, concept_synonym).

All functions accept a `DatabaseConnector` connection and a `vocabularyDatabaseSchema` argument, and return tidy tibbles with lowercase column names.

0.1 Connecting to a vocabulary

Any OMOP CDM connection works. The examples below use Eunomia as a small local SQLite vocabulary for illustration.

```
connection <- DatabaseConnector::connect(  
  Eunomia::getEunomiaConnectionDetails()  
)  
vocabularyDatabaseSchema <- "main"
```

For a full vocabulary you can point at a live CDM or a local SQLite file containing an Athena vocabulary download:

```
connectionDetails <- DatabaseConnector::createConnectionDetails(  
  dbms = "postgresql",  
  server = "your-server/cdm",  
  user = "user",  
  password = "password"  
)  
connection <- DatabaseConnector::connect(connectionDetails)  
vocabularyDatabaseSchema <- "cdm_vocab"
```

0.2 `searchConcepts()` — keyword search

`searchConcepts()` performs a case-insensitive partial match against `concept_name`. It is the simplest and most universally compatible search.

```
# Find standard condition concepts matching "atrial fibrillation"
results <- searchConcepts(
  "atrial fibrillation",
  connection,
  vocabularyDatabaseSchema = vocabularyDatabaseSchema
)
results
```

The returned tibble has columns: `concept_id`, `concept_name`, `domain_id`, `vocabulary_id`, `concept_class_id`, `standard_concept`, `concept_code`.

0.2.1 Filtering results

Use `domain` to restrict by OMOP domain and `standardOnly` to include non-standard concepts:

```
# Only drug concepts, include non-standard (source codes)
searchConcepts(
  "metformin",
  connection,
  vocabularyDatabaseSchema = vocabularyDatabaseSchema,
  domain = "Drug",
  standardOnly = FALSE,
  limit = 20L
)
```

0.3 rankedSearchConcepts() — ranked search

`rankedSearchConcepts()` uses a two-phase approach: a fast `ILIKE` scan to identify candidates, then similarity-based ranking. It also scores against synonyms and counts available source-to-standard mappings.

The ranking algorithm is automatically selected based on the connected DBMS:

DBMS	Similarity function
PostgreSQL	<code>pg_trgm similarity()</code> (requires the extension)
Snowflake	<code>JAROWINKLER_SIMILARITY()</code>
Spark / Databricks	<code>normalized levenshtein()</code>
All others	positional boost (exact > prefix > contains)

```
results <- rankedSearchConcepts(
  "atrial fibrillation",
  connection,
  vocabularyDatabaseSchema = vocabularyDatabaseSchema,
  limit = 20L
)
# Results include relevance score and mapping_count
results[, c("concept_id", "concept_name", "relevance", "mapping_count")]
```

The `relevance` column ranges from 0 to ~1.5 (higher is better). Use `offset` for pagination:

```
# Second page
rankedSearchConcepts(
  "atrial fibrillation",
  connection,
```

```

vocabularyDatabaseSchema = vocabularyDatabaseSchema,
limit = 20L,
offset = 0L
)

```

For PostgreSQL users: add a GIN trigram index for dramatically faster search:

```

CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX ON cdm_vocab.concept USING GIN (concept_name gin_trgm_ops);

```

0.4 getConceptDescendants() — concept hierarchy

getConceptDescendants() traverses concept_ancestor to return the full descendant tree of one or more seed concepts. This mirrors the descendants() modifier in cs(), but returns the full concept detail tibble so you can review what would be included before building a concept set.

```

# All descendants of "Atrial Fibrillation" (313217)
afib_desc <- getConceptDescendants(
  313217L,
  connection,
  vocabularyDatabaseSchema = vocabularyDatabaseSchema
)
afib_desc

```

The min_levels_of_separation column shows distance from the seed. Use minLevels and maxLevels to control depth:

```

# Direct children only (level 1)
getConceptDescendants(
  313217L,
  connection,
  vocabularyDatabaseSchema = vocabularyDatabaseSchema,
  minLevels = 1L,
  maxLevels = 1L
)

```

0.5 mapSourceToStandard() — source code lookup

mapSourceToStandard() maps non-standard source codes (ICD-9, ICD-10, NDC, etc.) to their standard OMOP concepts via the 'Maps to' relationship.

```

# Map ICD-10 atrial fibrillation codes to standard concepts
mapSourceToStandard(
  c("I48", "I48.0", "I48.1"),
  connection,
  vocabularyDatabaseSchema = vocabularyDatabaseSchema,
  vocabularyId = "ICD10CM"
)

```

Omit vocabularyId to search across all vocabularies.

0.6 getConceptInfo() — lookup by ID

When you already have concept IDs and want the full concept table details:

```
getConceptInfo(  
  c(313217L, 320128L, 201826L),  
  connection,  
  vocabularyDatabaseSchema = vocabularyDatabaseSchema  
)
```

0.7 End-to-end: search → build concept set → build cohort

The typical workflow is to use `searchConcepts()` or `rankedSearchConcepts()` to identify concept IDs interactively, then use those IDs in `cs()` to build a concept set for a cohort definition.

```
# 1. Find the right concept ID  
results <- rankedSearchConcepts(  
  "atrial fibrillation",  
  connection,  
  vocabularyDatabaseSchema = vocabularyDatabaseSchema,  
  domain = "Condition"  
)  
# Inspect results - concept 313217 is "Atrial fibrillation"  
  
# 2. Build a concept set with all descendants  
afib_cs <- cs(descendants(313217L), name = "Atrial Fibrillation")  
  
# 3. Optionally fill in display names for Atlas  
afib_cs <- getConceptSetDetails(  
  afib_cs,  
  connection,  
  vocabularyDatabaseSchema = vocabularyDatabaseSchema  
)  
  
# 4. Build the cohort  
afib_cohort <- cohort(  
  entry = entry(  
    conditionOccurrence(afib_cs),  
    primaryCriteriaLimit = "First"  
  ),  
  exit = exit(endStrategy = observationExit())  
)  
  
# 5. Serialize to JSON  
toCohortJson(afib_cohort)
```

```
DatabaseConnector::disconnect(connection)
```